

A Beginner's Guide to the Discrete Fourier Transform and Spectral Methods

For Readers of *Optimal Mixing of Passive Scalars*
Python Implementation and Numerical Results

Adebanji Adelowo

June 1, 2026

Contents

Why This Guide Exists	1
1 Part 1: The Big Picture — What Is a Spectral Method?	1
1.1 The Problem We Want to Solve	1
1.2 Two Ways to Represent a Function	2
2 Part 2: Waves and Frequencies	2
2.1 What Is a Frequency?	2
2.2 Any Periodic Function Is a Sum of Waves	2
2.3 Complex Exponentials — A Compact Notation	3
3 Part 3: The Discrete Fourier Transform (DFT)	3
3.1 From Continuous to Discrete	3
3.2 What Do the Fourier Coefficients Mean?	4
3.3 The FFT — The Practical Tool	4
3.4 The 2D DFT	5
4 Part 4: Wavenumbers — What Are They?	5
4.1 The Wavenumber Array	5
4.2 Why Negative Frequencies?	6
4.3 The Nyquist Frequency — The Highest Representable Frequency	6
4.4 The Nyquist Mode and Why It Is Zeroed for First Derivatives	7
5 Part 5: Spectral Differentiation — The Core Magic Trick	7
5.1 Why Differentiation Becomes Multiplication	7
5.2 Second Derivatives and the Laplacian	8
5.3 The Operators <code>DEL_X</code> and <code>DEL_Y</code> in the Code	9
6 Part 6: The Laplacian and Its Inverse	9

6.1	LAP_INV — The Inverse Laplacian	9
6.2	Why We Need Δ^{-1} for the Velocity	9
7	Part 7: Parseval’s Identity — Connecting Norms	10
7.1	The Statement	10
7.2	What It Means	10
7.3	How It Is Used in the Code	10
8	Part 8: The H^{-1} Mix Norm	11
8.1	Why Not Just Use the L^2 Norm?	11
8.2	The H^{-1} Norm	11
8.3	Intuition for the H^{-1} Norm	11
8.4	The λ^{-1} Weight	11
9	Part 9: The Leray Projection — Enforcing Incompressibility	12
9.1	What Is Incompressibility?	12
9.2	The Helmholtz Decomposition	12
9.3	In Fourier Space — It’s Just Multiplication	13
10	Part 10: The Pseudo-Spectral Method	13
10.1	The Problem with Products	13
10.2	The Pseudo-Spectral Solution	13
10.3	The 5-Step RHS Pipeline	13
11	Part 11: Time Integration with RK45	14
11.1	The ODE	14
11.2	What Is RK45?	14
11.3	Real/Imaginary Splitting	15
12	Part 12: The Resolution Check — When to Stop	15
12.1	Why the Simulation Must Stop	15
12.2	What Is Conserved?	15
12.3	The Event Function	16
13	Recommended Books and Resources	16
14	Summary Table	17

Why This Guide Exists

The Numerical Methods section (Section 3) of the optimal mixing report uses concepts from Fourier analysis and spectral methods — tools from signal processing and applied mathematics that can feel very abstract at first. This guide explains every concept from the ground up.

If you have studied calculus and basic linear algebra, you have everything you need to follow this guide.

1 Part 1: The Big Picture — What Is a Spectral Method?

1.1 The Problem We Want to Solve

The mixing simulation involves solving a partial differential equation (PDE):

$$\frac{\partial \theta}{\partial t} + u \cdot \nabla \theta = 0. \quad (1)$$

This describes how a scalar field $\theta(x, y, t)$ — think of it as the concentration of ink in water — changes over time under a velocity field u . The term $\nabla \theta$ means “gradient of θ ”: a vector pointing in the direction of steepest increase.

To solve this on a computer we must *discretise* it: approximate it on a finite grid of points. There are many ways to do this. The **spectral method** is one of the most accurate.

1.2 Two Ways to Represent a Function

Imagine a wiggly function $f(x)$ defined on $[0, 1]$:

Way 1 — Physical space. Store the values $f(x_0), f(x_1), \dots, f(x_{N-1})$ at grid points. Think of it as a bar chart of heights.

Way 2 — Frequency space (Fourier space). Represent f as a sum of sine and cosine waves of different frequencies. Instead of storing heights, store *how much of each frequency is present*.

These are just two descriptions of the same function — like describing a song by its audio waveform versus its sheet music. You can switch between them freely.

Key insight: Some operations that are hard in physical space (like computing derivatives) become trivially simple in frequency space — just multiply by a number.

2 Part 2: Waves and Frequencies

2.1 What Is a Frequency?

A pure wave of frequency k on $[0, 1]$ looks like:

$$f_k(x) = \sin(2\pi kx) \quad \text{or} \quad f_k(x) = \cos(2\pi kx).$$

The number k counts how many complete oscillations fit inside $[0, 1]$:

- $k = 1$: one full oscillation — a single gentle curve.
- $k = 2$: two full oscillations — a slightly wigglier curve.
- $k = 10$: ten oscillations — a rapidly oscillating wave.

Higher k = higher frequency = finer (smaller-scale) features.

2.2 Any Periodic Function Is a Sum of Waves

This is the central claim of Fourier analysis: **any periodic function can be built by adding up sine and cosine waves**. This sum is the **Fourier series**:

$$f(x) = a_0 + \sum_{k=1}^{\infty} [a_k \cos(2\pi kx) + b_k \sin(2\pi kx)]. \quad (2)$$

The coefficients a_k and b_k tell you how much of each frequency is present.

A concrete example. A square wave (alternating $+1$ and -1) can be written as:

$$f(x) = \frac{4}{\pi} \left[\sin(2\pi x) + \frac{1}{3} \sin(6\pi x) + \frac{1}{5} \sin(10\pi x) + \dots \right].$$

Adding the first few terms already gives a recognisable approximation; the infinite series is exact.

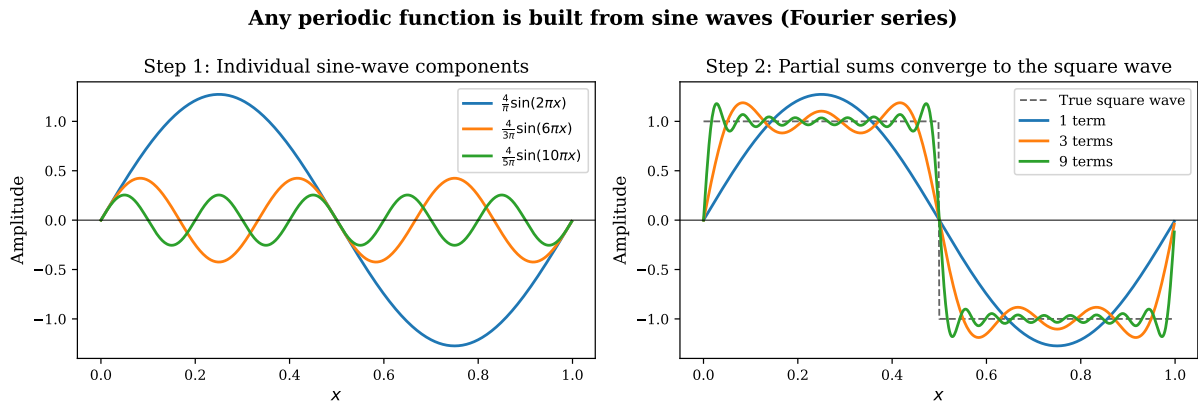


Figure 1: Left: the first three sine-wave components of the square wave’s Fourier series. Right: adding more and more terms (1, 3, 9) progressively improves the approximation. The Gibbs overshoot near the jumps shrinks as more terms are included but never entirely disappears.

2.3 Complex Exponentials — A Compact Notation

Instead of writing sin and cos separately, mathematicians use **complex exponentials** via Euler’s formula:

$$e^{2\pi i kx} = \cos(2\pi kx) + i \sin(2\pi kx).$$

The Fourier series then takes the symmetric form:

$$f(x) = \sum_{k=-\infty}^{\infty} \hat{f}(k) e^{2\pi i kx}. \quad (3)$$

The complex number $\hat{f}(k)$ is the **Fourier coefficient** at wavenumber k . Its magnitude $|\hat{f}(k)|$ is the amplitude of that frequency, and its argument $\arg(\hat{f}(k))$ is the phase shift.

3 Part 3: The Discrete Fourier Transform (DFT)

3.1 From Continuous to Discrete

The Fourier series works for continuous functions. Computers work with finite lists of numbers. We need a discrete version.

Suppose we have N equally spaced sample points:

$$x_j = \frac{j}{N}, \quad j = 0, 1, 2, \dots, N-1.$$

The **Discrete Fourier Transform (DFT)** takes the N values $f[0], f[1], \dots, f[N-1]$ and produces N Fourier coefficients:

$$\hat{f}[k] = \sum_{j=0}^{N-1} f[j] e^{-2\pi i j k / N}, \quad k = 0, 1, \dots, N-1. \quad (4)$$

The **Inverse DFT (IDFT)** goes back:

$$f[j] = \frac{1}{N} \sum_{k=0}^{N-1} \hat{f}[k] e^{2\pi i j k / N}. \quad (5)$$

No information is lost. Going forward and back is exact. The DFT and IDFT are just two different representations of the same data — like converting between Celsius and Fahrenheit.

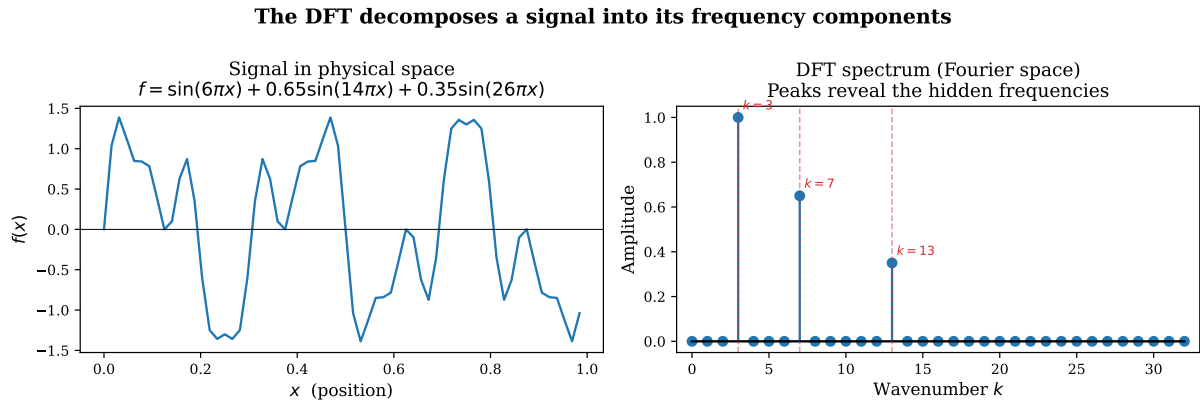


Figure 2: Left: a signal composed of three sine waves at $k = 3, 7$, and 13 . Right: the magnitude of its DFT reveals exactly those three frequencies as sharp peaks — the DFT has “heard” each component separately, even though they are all jumbled together in physical space.

3.2 What Do the Fourier Coefficients Mean?

$\hat{f}[k]$ is a complex number whose:

- **Magnitude** $|\hat{f}[k]|$ tells you how much of frequency k is present in f .

- **Phase** $\arg(\hat{f}[k])$ tells you how that wave is shifted (its offset).

If f is real-valued (as physical quantities always are), the negative-frequency coefficients are redundant: $\hat{f}[-k] = \overline{\hat{f}[k]}$ (complex conjugate).

3.3 The FFT — The Practical Tool

Computing the DFT directly requires $O(N^2)$ operations — for $N = 1000$, that is 10^6 multiplications. The **Fast Fourier Transform (FFT)** algorithm computes the *exact same result* in $O(N \log N)$ operations — for $N = 1000$, that is only $\approx 10,000$.

FFT vs. DFT: the algorithm is different, the result is identical.

```

1 f_hat = np.fft.fft(f)           # 1D FFT
2 f_back = np.fft.ifft(f_hat)     # 1D inverse FFT
3
4 F_hat = np.fft.fft2(F)          # 2D FFT (applied to each axis)
5 F_back = np.fft.ifft2(F_hat)    # 2D inverse FFT

```

3.4 The 2D DFT

For a 2D function $f(x, y)$ defined on an $N \times N$ grid, the DFT is applied in both directions independently:

$$\hat{f}[k_y, k_x] = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} f[i, j] e^{-2\pi i(ik_y + jk_x)/N}. \quad (6)$$

Index convention used in this project (and NumPy).

- Row index i corresponds to the **y -direction**.
- Column index j corresponds to the **x -direction**.
- So `theta_hat[ky, kx]` is the coefficient for wavenumber (k_x, k_y) .

This is why `DEL_X` has shape $(1, N)$ (acts on columns) and `DEL_Y` has shape $(N, 1)$ (acts on rows).

4 Part 4: Wavenumbers — What Are They?

4.1 The Wavenumber Array

After computing `fft(f)` on an array of length N , the output has N entries corresponding to N different frequencies stored in a specific order.

For $N = 8$, the array indices and their corresponding effective wavenumbers are:

Index	0	1	2	3	4	5	6	7
Wavenumber	0	1	2	3	± 4	-3	-2	-1

- **Index 0:** the constant (DC) component, $k = 0$ — the average value of f .

- **Indices 1 to $N/2 - 1$:** positive frequencies.
- **Index $N/2$:** the Nyquist frequency $k = N/2$ (special case, see below).
- **Indices $N/2 + 1$ to $N - 1$:** negative frequencies.

In the code:

```
1 k = np.arange(N)
2 k_eff = k - N * (k > N // 2) # convert index to effective
   wavenumber
```

This is called “centring” the wavenumbers. To visualise a 2D spectrum with zero in the middle, use `np.fft.fftshift`.

How NumPy stores wavenumbers in the FFT output ($N = 16$ points)

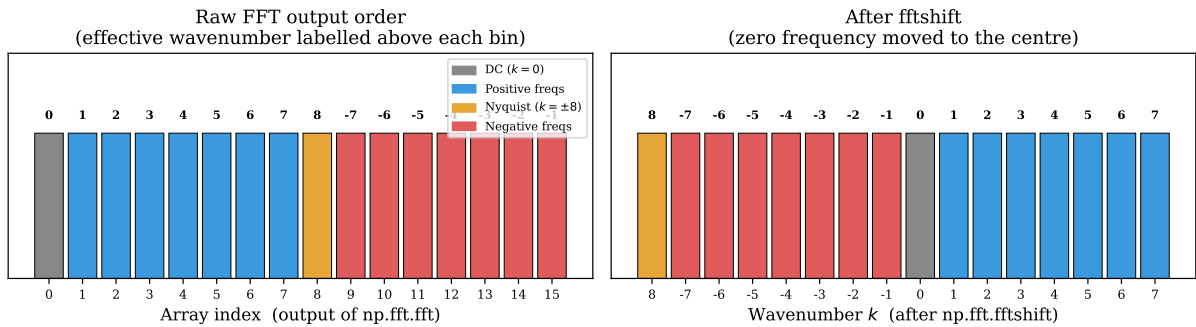


Figure 3: Wavenumber storage for $N = 16$. Left: the raw output of `np.fft.fft`. Indices 0–7 hold positive frequencies; index 8 is the Nyquist; indices 9–15 hold negative frequencies (wrapped around). Right: after `np.fft.fftshift` the zero-frequency bin sits in the middle, which is more intuitive for visualisation.

4.2 Why Negative Frequencies?

Negative frequencies arise naturally from the complex exponential formulation. For a real-valued function f , the k and $-k$ modes always come in complex-conjugate pairs. Together they produce a real-valued signal:

$$\hat{f}[k] e^{2\pi i k x} + \hat{f}[-k] e^{-2\pi i k x} = 2|\hat{f}[k]| \cos(2\pi k x + \phi).$$

So negative frequencies represent the “mirror image” of positive frequencies — they are not physically different, just part of the mathematical convention.

4.3 The Nyquist Frequency — The Highest Representable Frequency

The grid has spacing $dx = 1/N$. The finest feature representable is a wave that alternates sign at every grid point — one oscillation per $2dx$. That frequency is:

$$k_{\text{Nyquist}} = \frac{N}{2}.$$

You cannot represent frequencies higher than $N/2$ on a grid with N points. If a function has fine-scale features at $k > N/2$, the DFT *cannot see them* — they get wrapped around and appear as lower-frequency noise. This is called **aliasing**.

Real-world analogy. A movie camera shoots at 24 frames per second. A car wheel spinning at more than 12 revolutions per second appears to spin *backwards* in the footage — this is aliasing. The camera cannot represent frequencies faster than 12 Hz.

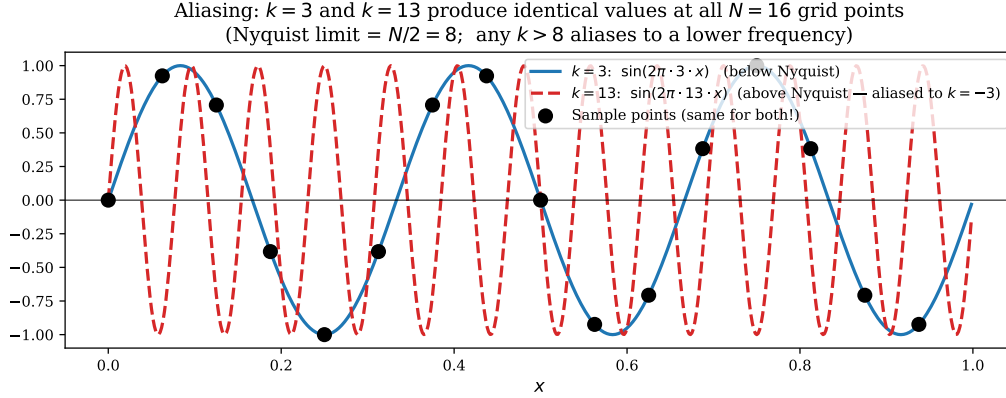


Figure 4: Aliasing on a grid with $N = 16$ points (Nyquist limit = 8). The wave $k = 3$ (solid blue) and the wave $k = 13$ (dashed red) produce *identical* values at every grid point (filled circles), so the DFT cannot distinguish them. Any frequency above the Nyquist is silently folded back into a lower frequency.

4.4 The Nyquist Mode and Why It Is Zeroed for First Derivatives

At $k = N/2$, the wave $e^{2\pi i(N/2)x}$ sampled at grid points $x_j = j/N$ gives:

$$e^{2\pi i(N/2)(j/N)} = e^{\pi i j} = (-1)^j.$$

This alternates $+1, -1, +1, -1, \dots$ — the same as $e^{-2\pi i(N/2)x}$. So $k = +N/2$ and $k = -N/2$ are **identical on the grid** — they are ambiguous.

- For the **Laplacian** (second derivative), this is fine: $(-k)^2 = k^2$, so the sign does not matter.
- For **first derivatives**, the sign matters: $+k$ and $-k$ give derivatives with opposite signs. Since the mode is ambiguous, we zero it out:

```
1 k_d = k_eff.copy()
2 k_d[N//2] = 0.0    # zero the Nyquist mode for first derivatives
```

DEL_X and DEL_Y use k_d; LAP_INV uses k_eff.

5 Part 5: Spectral Differentiation — The Core Magic Trick

5.1 Why Differentiation Becomes Multiplication

This is the single most important idea in spectral methods. Consider a function written as a sum of waves:

$$f(x) = \sum_k \hat{f}(k) e^{2\pi i k x}.$$

Take the derivative term by term:

$$\frac{df}{dx} = \sum_k \hat{f}(k) \cdot \frac{d}{dx} [e^{2\pi i k x}] = \sum_k \hat{f}(k) \cdot (2\pi i k) e^{2\pi i k x}.$$

In Fourier space, taking a derivative simply means multiplying each coefficient by $2\pi i k$.

Recipe for computing df/dx spectrally.

1. Compute $\hat{f} = \text{FFT}(f)$.
2. Multiply: $\widehat{df/dx}[k] = (2\pi i k) \hat{f}[k]$.
3. Transform back: $df/dx = \text{real}(\text{IFFT}(\widehat{df/dx}))$.

```
1 f_hat = np.fft.fft(f)
2 df_hat = (2j * np.pi * k_d) * f_hat      # k_d has Nyquist zeroed
3 df_dx = np.real(np.fft.ifft(df_hat))      # discard tiny numerical
      imaginary part
```

Accuracy. Spectral differentiation is **spectrally accurate** — the error decreases faster than any polynomial in N as $N \rightarrow \infty$ (for smooth periodic functions). It is far more accurate than finite differences, which are only $O(dx^p)$ accurate for some fixed order p .

Spectral differentiation: the accuracy advantage

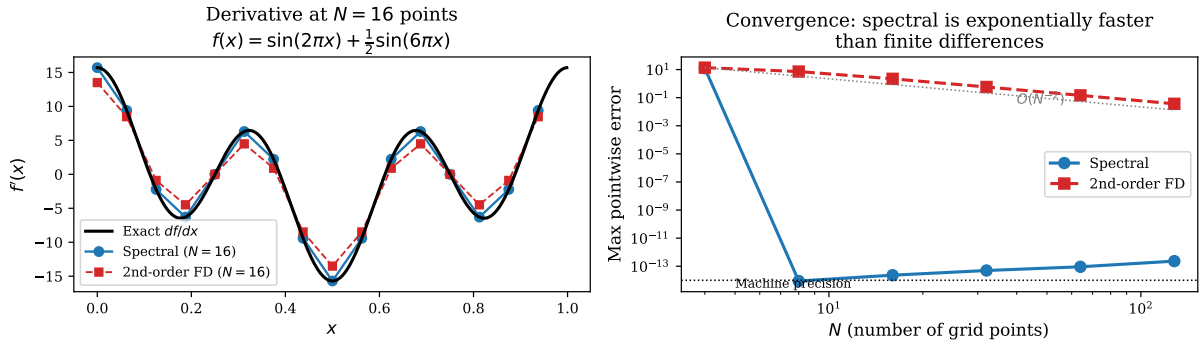


Figure 5: Left: spectral differentiation (blue) matches the exact derivative perfectly at $N = 16$, while second-order finite differences (red) show visible error. Right: the convergence plot shows that spectral errors drop to machine precision by $N = 16$ for this smooth function, whereas finite-difference errors decay only as $O(N^{-2})$. For rough functions the spectral advantage is smaller, but for the smooth fields in the mixing simulation it is enormous.

5.2 Second Derivatives and the Laplacian

Two derivatives each contribute a factor of $2\pi i k$:

$$(2\pi i k)^2 = -4\pi^2 k^2.$$

So the second-derivative multiplier is $-4\pi^2 k^2$ (real and negative).

In 2D, the Laplacian $\Delta f = \partial_{xx} f + \partial_{yy} f$ becomes:

$$\widehat{\Delta f}[k_y, k_x] = [-(2\pi k_x)^2 - (2\pi k_y)^2] \hat{f}[k_y, k_x] = -4\pi^2(k_x^2 + k_y^2) \hat{f}[k_y, k_x]. \quad (7)$$

5.3 The Operators DEL_X and DEL_Y in the Code

```
1 DEL_X = (2j * np.pi * k_d)[np.newaxis, :] # shape (1, N)
2 DEL_Y = (2j * np.pi * k_d)[:, np.newaxis] # shape (N, 1)
```

For a 2D field θ :

- `DEL_X * theta_hat` multiplies each *column* (x -direction) by the corresponding $2\pi i k_x$ — giving $\widehat{\partial_x \theta}$.
- `DEL_Y * theta_hat` multiplies each *row* (y -direction) by $2\pi i k_y$ — giving $\widehat{\partial_y \theta}$.

Why shapes (1, N) and (N, 1)? NumPy broadcasting means (1, N) is automatically repeated along rows to become (N, N) when multiplied with an (N, N) array. This saves memory without storing any redundant data.

6 Part 6: The Laplacian and Its Inverse

6.1 LAP_INV — The Inverse Laplacian

We established that the Laplacian in Fourier space is multiplication by $-4\pi^2 |k|^2$. The **inverse Laplacian** is therefore division by the same factor:

$$\text{LAP_INV}[k_y, k_x] = \begin{cases} \frac{-1}{(2\pi)^2(k_x^2 + k_y^2)}, & (k_x, k_y) \neq (0, 0), \\ 0, & (k_x, k_y) = (0, 0). \end{cases} \quad (8)$$

To apply Δ^{-1} to a field θ :

1. Compute $\hat{\theta} = \text{FFT2}(\theta)$.
2. Multiply: $\widehat{\Delta^{-1}\theta} = \text{LAP_INV} \cdot \hat{\theta}$.
3. Transform back: $\Delta^{-1}\theta = \text{real}(\text{IFFT2}(\widehat{\Delta^{-1}\theta}))$.

Why zero the DC mode? At $k = (0, 0)$ we would be dividing by zero. The DC mode represents the spatial mean $\langle \theta \rangle = \int \theta dx dy$. This mean is conserved and decoupled from the rest of the dynamics, so we set it to zero.

Physical interpretation of Δ^{-1} . The inverse Laplacian **smooths** a function. High-frequency (small-scale) features are suppressed by the large $|k|^2$ in the denominator. If θ is a concentrated blob, $\Delta^{-1}\theta$ is a broad, smooth distribution.

6.2 Why We Need Δ^{-1} for the Velocity

In the LTD optimal velocity formula:

$$v = -\Delta^{-1}P(\theta \nabla \Delta^{-1}\theta),$$

the Δ^{-1} appears **twice**:

1. The inner $\Delta^{-1}\theta$ converts the scalar into a stream-function-like field (smoother, large-scale).
2. The outer $-\Delta^{-1}$ converts the nonlinear term into the velocity itself.

This ensures the velocity has the right structure: divergence-free with controlled enstrophy.

7 Part 7: Parseval's Identity — Connecting Norms

7.1 The Statement

For a 1D DFT with N points:

$$\sum_{j=0}^{N-1} |f[j]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |\hat{f}[k]|^2.$$

In the NumPy convention (non-unitary), $\|\hat{f}\|^2 = N\|f\|^2$, so:

$$\|f\|_{L^2} \approx \frac{\|\hat{f}\|_{\ell^2}}{N}.$$

In 2D (with N^2 points):

$$\|f\|_{L^2} \approx \frac{\|\hat{f}\|_{\ell^2}}{N^2}. \quad (9)$$

7.2 What It Means

Parseval's identity says: **the total energy (sum of squares) is the same in physical space and Fourier space**, up to a normalisation factor.

This is analogous to the Pythagorean theorem: if you rotate a vector, its length does not change. The DFT is a rotation in an N -dimensional space, so it preserves lengths.

7.3 How It Is Used in the Code

To compute $\|\nabla v\|_{L^2}$ (needed for the enstrophy normalisation), the code computes in Fourier space:

```
1 grad_v_norm = (1/N**2) * np.sqrt(  
2     np.sum(np.abs(DEL_X * v_x_hat)**2) +  
3     np.sum(np.abs(DEL_Y * v_x_hat)**2) +  
4     np.sum(np.abs(DEL_X * v_y_hat)**2) +  
5     np.sum(np.abs(DEL_Y * v_y_hat)**2)  
6 )
```

Each `DEL_X * v_x_hat` is $\widehat{\partial_x v_x}$ in Fourier space, and the Parseval factor $1/N^2$ converts the Fourier-space norm to the physical-space L^2 norm.

8 Part 8: The H^{-1} Mix Norm

8.1 Why Not Just Use the L^2 Norm?

Under the passive scalar equation $\partial_t \theta + u \cdot \nabla \theta = 0$ with divergence-free u , all L^p norms are **conserved**:

$$\|\theta(t)\|_{L^2} = \|\theta_0\|_{L^2} \quad \text{for all } t.$$

So the L^2 norm cannot tell us anything about mixing — it is constant regardless of how well the dye has been stirred. We need a norm that *decreases* as the scalar gets mixed into finer and finer structures.

8.2 The H^{-1} Norm

The H^{-1} **norm** is defined as:

$$\|\theta\|_{H^{-1}}^2 = \sum_{k \neq 0} \frac{|\hat{\theta}(k)|^2}{4\pi^2 |k|^2}. \quad (10)$$

Compare with the L^2 norm: $\|\theta\|_{L^2}^2 = \sum_k |\hat{\theta}(k)|^2$.

The difference: the H^{-1} norm divides each mode's contribution by $|k|^2$, giving **much less weight to high-frequency (fine-scale) content**.

What happens when we stir? Initially, the scalar's energy is concentrated at low wavenumbers (smooth, large blobs). As the velocity stirs it, energy is transferred to higher and higher wavenumbers (finer filaments). The H^{-1} norm penalises low- k content heavily — so as energy moves to high k , the norm decreases.

8.3 Intuition for the H^{-1} Norm

Think of it this way:

- A scalar with all its energy at low k (one big blob) has **large** H^{-1} norm — it is unmixed.
- A scalar with its energy spread uniformly across many k values has **small** H^{-1} norm — it is well-mixed.
- A perfectly uniform scalar ($\theta = \text{const}$, all energy at $k = 0$) has zero H^{-1} norm — perfectly mixed.

8.4 The λ^{-1} Weight

In code, the mix norm is computed via `LAMBDA_INV`:

$$\text{LAMBDA_INV}[k] = \frac{1}{2\pi|k|} = \sqrt{-\text{LAP_INV}[k]}, \quad (11)$$

$$\|\theta\|_{H^{-1}}^2 = \frac{\|\text{LAMBDA_INV} \cdot \hat{\theta}\|_{\ell^2}^2}{N^4}. \quad (12)$$

```

1 LAMBDA_INV = np.sqrt(np.where(LAP_INV != 0, -LAP_INV, 0.0))
2 mix_norm_sq = np.sum(np.abs(LAMBDA_INV * theta_hat)**2) / N**4

```

The H^{-1} mix norm decreases as fine-scale structure develops
(Both fields have the same L^2 norm ≈ 0.153)

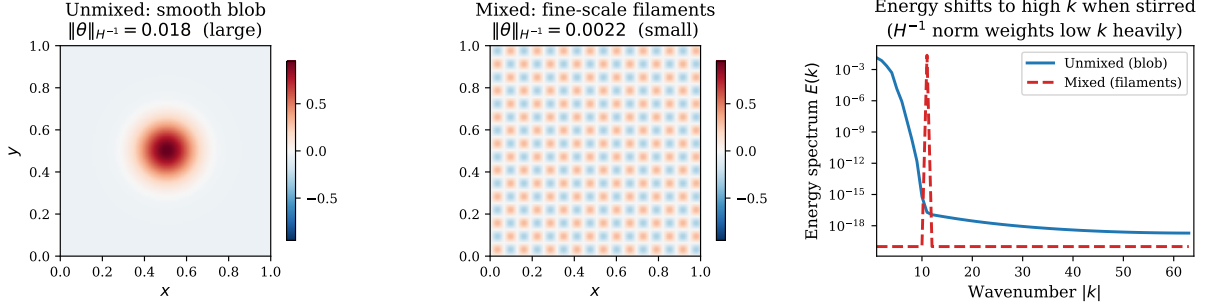


Figure 6: The H^{-1} norm as a measure of mixing quality. Left: an unmixed Gaussian blob — all energy concentrated at low wavenumbers, $\|\theta\|_{H^{-1}}$ is large. Centre: a fine-scale checkerboard pattern with the *same* L^2 norm — energy spread to high wavenumbers, $\|\theta\|_{H^{-1}}$ is much smaller. Right: the energy spectrum confirms the shift from low- k to high- k content. As the mixing simulation runs, the H^{-1} norm decreases monotonically.

9 Part 9: The Leray Projection — Enforcing Incompressibility

9.1 What Is Incompressibility?

The velocity field $u = (u_x, u_y)$ must satisfy $\nabla \cdot u = 0$:

$$\frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y} = 0.$$

This is the **incompressibility condition**: fluid is not created or destroyed anywhere. In physical terms, any fluid flowing into a region must flow out equally.

9.2 The Helmholtz Decomposition

Any vector field g can be split into two parts:

$$g = \underbrace{g_{\text{div-free}}}_{\nabla \cdot g_{\text{div-free}} = 0} + \underbrace{\nabla \phi}_{\text{pure gradient}},$$

where ϕ is some scalar potential. The **Leray projection** extracts just the divergence-free part:

$$Pg = g - \nabla \phi, \quad \phi = \Delta^{-1}(\nabla \cdot g).$$

Step by step:

1. Compute the divergence: $D = \nabla \cdot g = \partial_x g_x + \partial_y g_y$.

2. Solve $\Delta\phi = D$, i.e. $\phi = \Delta^{-1}D$.
3. Subtract the gradient: $Pg = g - \nabla\phi$.

9.3 In Fourier Space — It's Just Multiplication

Everything becomes element-wise multiplication in Fourier space:

$$\widehat{Pg}_x[k] = \hat{g}_x[k] - \text{DEL_X}[k] \cdot \text{LAP_INV}[k] \cdot (\text{DEL_X}[k] \cdot \hat{g}_x[k] + \text{DEL_Y}[k] \cdot \hat{g}_y[k]). \quad (13)$$

In code:

```

1 div_g_hat = DEL_X * gx_hat + DEL_Y * gy_hat      # divergence
  of g
2 Pgx_hat   = gx_hat - DEL_X * LAP_INV * div_g_hat  # project x-
  component
3 Pgy_hat   = gy_hat - DEL_Y * LAP_INV * div_g_hat  # project y-
  component

```

No iteration required — just a handful of array multiplications.

10 Part 10: The Pseudo-Spectral Method

10.1 The Problem with Products

A purely spectral method would compute *everything* in Fourier space. But the term $\theta \cdot \nabla \Delta^{-1} \theta$ involves a **product of two functions in physical space**.

In Fourier space, a product becomes a **convolution**:

$$\widehat{f \cdot g}[k] = \sum_m \hat{f}[m] \hat{g}[k - m].$$

Computing this naively costs $O(N^2)$ per wavenumber — far more expensive than FFTs.

10.2 The Pseudo-Spectral Solution

The **pseudo-spectral trick**: compute products in physical space (cheap, pointwise), and everything else in Fourier space:

1. Transform the fields you need from Fourier space to physical space (IFFT).
2. Compute the product pointwise (simple array multiplication).
3. Transform the result back to Fourier space (FFT).

Each FFT/IFFT costs $O(N^2 \log N)$ in 2D. The total cost is much lower than convolution.

10.3 The 5-Step RHS Pipeline

Every time the ODE solver requests $d\hat{\theta}/dt$, the following five steps run.

Step 1 — Compute $g = \theta \nabla \Delta^{-1} \theta$ (pseudo-spectral).

- a. Apply Δ^{-1} in Fourier space: $\hat{\psi} = \text{LAP_INV} \cdot \hat{\theta}$.
- b. Compute $\psi = \text{real}(\text{IFFT2}(\hat{\psi}))$ (physical space stream-function-like field).
- c. Compute derivatives: $\partial_x \psi = \text{real}(\text{IFFT2}(\text{DEL_X} \cdot \hat{\psi}))$ and $\partial_y \psi$ similarly.
- d. Compute the nonlinear product in physical space: $g_x = \theta \cdot \partial_x \psi$, $g_y = \theta \cdot \partial_y \psi$.
- e. Transform back: $\hat{g}_x = \text{FFT2}(g_x)$, $\hat{g}_y = \text{FFT2}(g_y)$.

Step 2 — Apply Leray projection (pure Fourier-space multiplication).

$$(\hat{g}_x, \hat{g}_y) \longrightarrow (\widehat{Pg_x}, \widehat{Pg_y}).$$

Step 3 — Apply $-\Delta^{-1}$ to get velocity $v = -\Delta^{-1}Pg$.

$$\hat{v}_x = -\text{LAP_INV} \cdot \widehat{Pg_x}, \quad \hat{v}_y = -\text{LAP_INV} \cdot \widehat{Pg_y}.$$

Step 4 — Normalise to enforce the enstrophy constraint $\|\nabla u\|_{L^2} = F$ (via Parseval).

$$\|\nabla v\|_{L^2} = \frac{1}{N^2} \sqrt{\|\text{DEL_X} \hat{v}_x\|^2 + \|\text{DEL_Y} \hat{v}_x\|^2 + \|\text{DEL_X} \hat{v}_y\|^2 + \|\text{DEL_Y} \hat{v}_y\|^2}, \quad \hat{u} = F \frac{\hat{v}}{\|\nabla v\|_{L^2}}.$$

Step 5 — Advect to compute $d\hat{\theta}/dt = -\mathcal{F}[u \cdot \nabla \theta]$.

$$\frac{d\hat{\theta}}{dt} = -\text{FFT2} \left[u_x \cdot \text{IFFT2}(\text{DEL_X} \cdot \hat{\theta}) + u_y \cdot \text{IFFT2}(\text{DEL_Y} \cdot \hat{\theta}) \right].$$

11 Part 11: Time Integration with RK45

11.1 The ODE

After spectral discretisation, the PDE becomes a large system of ODEs:

$$\frac{d\hat{\theta}}{dt} = F(\hat{\theta}, t), \quad \hat{\theta}(0) = \hat{\theta}_0, \tag{14}$$

where F is the RHS from the five steps above and $\hat{\theta}$ has N^2 complex components.

11.2 What Is RK45?

The **Runge–Kutta 4/5** (Dormand–Prince) method is a classical adaptive ODE solver. Here is the idea:

- At each time step t_n , evaluate F at several cleverly chosen intermediate points between t_n and $t_n + h$.
- Combine these evaluations to get a 4th-order *and* a 5th-order estimate of $\hat{\theta}(t_n + h)$.

- Use the difference between these two estimates to measure the local error.
- If the error is too large, halve the step size h and retry; if it is very small, increase h to speed up.

This is called an **adaptive** time integrator — it automatically picks a step size that balances accuracy and speed.

Why RK45 and not simpler methods? Euler’s method (first order) is too inaccurate. Fourth-order Runge–Kutta with a fixed step is accurate but wastes evaluations when the solution is smooth. RK45 is the standard workhorse for stiff-to-moderate ODEs and matches MATLAB’s `ode45`.

```

1 sol = scipy.integrate.solve_ivp(
2     rhs,
3     t_span=(0, T),
4     y0=theta_hat_flat_real,
5     method='RK45',
6     rtol=1e-6,
7     atol=1e-8
8 )

```

11.3 Real/Imaginary Splitting

`solve_ivp` requires a real-valued state vector. The complex array $\hat{\theta} \in \mathbb{C}^{N^2}$ is stored as:

$$y = [\text{Re}(\hat{\theta}_{\text{flat}}), \text{Im}(\hat{\theta}_{\text{flat}})] \in \mathbb{R}^{2N^2}.$$

At each RHS call, the code reconstructs $\hat{\theta}$ from y , performs the five-step computation, and returns dy/dt in real form.

12 Part 12: The Resolution Check — When to Stop

12.1 Why the Simulation Must Stop

As the velocity stirs θ into finer and finer filaments, the scalar develops features at increasingly high wavenumbers. Eventually these features become smaller than the grid spacing $dx = 1/N$ — the grid can no longer represent them. Beyond this point, the simulation is no longer physically meaningful.

12.2 What Is Conserved?

Under exact incompressible advection, **all L^p norms are conserved**:

$$\|\theta(t)\|_{L^p} = \|\theta_0\|_{L^p} \quad \text{for all } t \geq 0, \text{ all } p \geq 1.$$

This is a mathematical fact following from the change-of-variables formula for divergence-free flows. Numerically, these norms should remain constant as long as the grid can resolve the solution.

12.3 The Event Function

The code monitors:

$$e(t) = \max\left(\left|\|\theta\|_{L^2} - 1\right|, \left|\frac{\|\theta\|_{L^4}}{\|\theta_0\|_{L^4}} - 1\right|, \left|\frac{\|\theta\|_{L^8}}{\|\theta_0\|_{L^8}} - 1\right|\right) - \varepsilon, \quad \varepsilon = 10^{-3}. \quad (15)$$

When $e(t) > 0$ (any norm drifts by more than 0.1%), the solver stops.

Why also use L^4 and L^8 ? The L^2 norm is quadratic and relatively forgiving. Higher L^p norms are more sensitive to large values and sharp gradients — exactly the kind of structures that appear when the grid is under-resolving. Using three norms gives an earlier and more reliable warning.

13 Recommended Books and Resources

Introductory — Start Here

“A Student’s Guide to Fourier Transforms” J. F. James, Cambridge University Press, 3rd ed., 2011.

The most accessible introduction. Written specifically for science students without a heavy maths background. Short chapters, physical intuition, minimal prerequisites.

“The Fourier Transform and Its Applications” Ronald Bracewell, McGraw-Hill, 3rd ed., 2000.

A classic with excellent hand-drawn visualisations. Bracewell’s “picket fence” picture of the DFT and his sampling-theorem explanations are famous for being unusually clear.

“Fourier Analysis: An Introduction” Elias Stein & Rami Shakarchi, Princeton Lectures in Analysis, Vol. 1, 2003.

More rigorous, but very clearly written. Good for understanding *why* the theorems hold, not just how to use them.

Spectral / Numerical Methods — Directly Relevant

“Spectral Methods in MATLAB” Lloyd N. Trefethen, SIAM, 2000. *Free PDF online.*

The perfect companion to this project. 40 short programs (easily translated from MATLAB to Python) explaining spectral methods for PDEs. Very readable; starts from scratch. This book will make Section 3 of the report completely transparent.

“Numerical Methods for Engineers” Chapra & Canale, McGraw-Hill, 7th ed., 2015. Covers FFT, ODE solvers (including RK45), and numerical differentiation at an introductory engineering level. Very practical and hands-on.

“A Practical Guide to Pseudospectral Methods” Bengt Fornberg, Cambridge University Press, 1996.

Specifically about pseudo-spectral methods — exactly the approach used in the mixing simulation. More technical than Trefethen’s book, but the most thorough treatment of the subject.

“Numerical Methods for Fluid Dynamics” Dale Durran, Springer, 2nd ed., 2010.
Covers spectral methods, pseudo-spectral methods, and time integration at a graduate level. Bridges the gap between numerical methods and physical simulation.

PDEs and the Physical Problem

“Introduction to Partial Differential Equations” Walter Strauss, Wiley, 2nd ed., 2008.

Good introductory PDE text covering Fourier series and transforms from a mathematical perspective. Accessible to anyone with calculus.

“An Introduction to Fluid Dynamics” G.K. Batchelor, Cambridge University Press, 2000.

The definitive fluid dynamics reference. Chapters 1–3 are accessible and explain divergence-free velocity fields, the incompressibility condition, and passive scalar transport.

Online Resources

Resource	What It Covers
3Blue1Brown “But What Is the Fourier Transform?” (YouTube)	Best 20-min visual intro to Fourier
3Blue1Brown “But What Is a Fourier Series?” (YouTube)	Visual intuition for decomposing f
NumPy FFT documentation	Exact conventions used in <code>np.fft</code>
Trefethen’s ATAP (free PDF)	Rigorous approximation theory, sp
SciPy <code>solve_ivp</code> documentation	Full details of the RK45 implemen

14 Summary Table

Concept	What It Is	Code Equivalent
DFT	Decompose discrete function into frequencies	<code>np.fft.fft2</code>
FFT	Fast $O(N \log N)$ algorithm for DFT	<code>np.fft.fft2</code>
Wavenumber k	Frequency index; oscillations per unit length	<code>k_eff</code>
Centred wavenumbers	Remap DFT indices to $\{-N/2, \dots, N/2\}$	<code>k - N*(k > N//2)</code>
Nyquist frequency	Maximum representable frequency = $N/2$	Index $N//2$
Spectral derivative	Multiply $\hat{f}$ by $2\pi i k$	<code>DEL_X, DEL_Y</code>
Inverse Laplacian	Multiply $\hat{f}$ by $-1/(4\pi^2 \ k\ ^2)$	<code>LAP_INV</code>
Parseval identity	$\ f\ _{L^2} = \ \hat{f}\ _{\ell^2}/N^2$ (2D)	Norm computations
H^{-1} mix norm	Weighted norm that decreases with mixing	<code>LAMBDA_INV</code>
Leray projection	Extracts divergence-free part of vector field	Step 2 of RHS
Pseudo-spectral	Products in physical, derivatives in Fourier	RHS evaluation
RK45	Adaptive-step 4/5 Runge–Kutta ODE solver	<code>solve_ivp</code>
Resolution check	Stop when L^p norms drift beyond tolerance	<code>make_res_check()</code>

This guide accompanies the notebook `00_dft_for_beginners.ipynb` in the `python_code/` folder, which provides interactive Python demonstrations of every concept described here.